ARM COMPILER

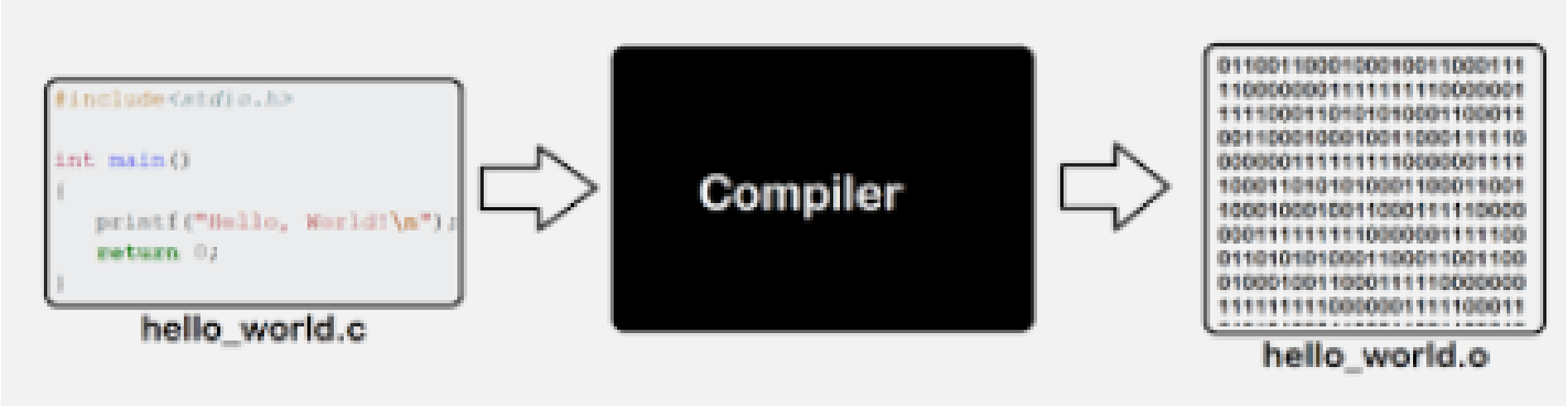
10 May 2023

OUTLINE

1. Overview Compiler and Arm Compiler
2. Armclang
3. Armlink
4. Fromelf

1. Overview Compiler and Arm Compiler

- The compiler is software that converts a program written in a high-level language (Source Language) to a low-level language (Object/Target/Machine Language/0's, 1's).
- Arm Compiler comprises tools to create ELF object files, ELF image files, and library files. Arm compiler can modify ELF object and image files, and display information on those files.
- Arm Compiler is a component of *Arm® Development Studio*, *Arm® DS-5 Development Studio*, and *Arm® Keil® MDK*. Alternatively, using Arm Compiler as a *standalone product*.



1. Supported C and C++ source language

Standard C	GNU C	Standard C++	GNU C++
c90	gnu90	c++98	gnu++98
c99	gnu99	c++03	gnu++03
c11 [COMMUNITY]	gnu11 [COMMUNITY]	c++11	gnu++11
-	-	c++14	gnu++14
-	-	c++17	gnu++17

1. Components in Arm Compiler

armclang	armlink	armar
armasm	fromelf	Arm C/C++ libraries

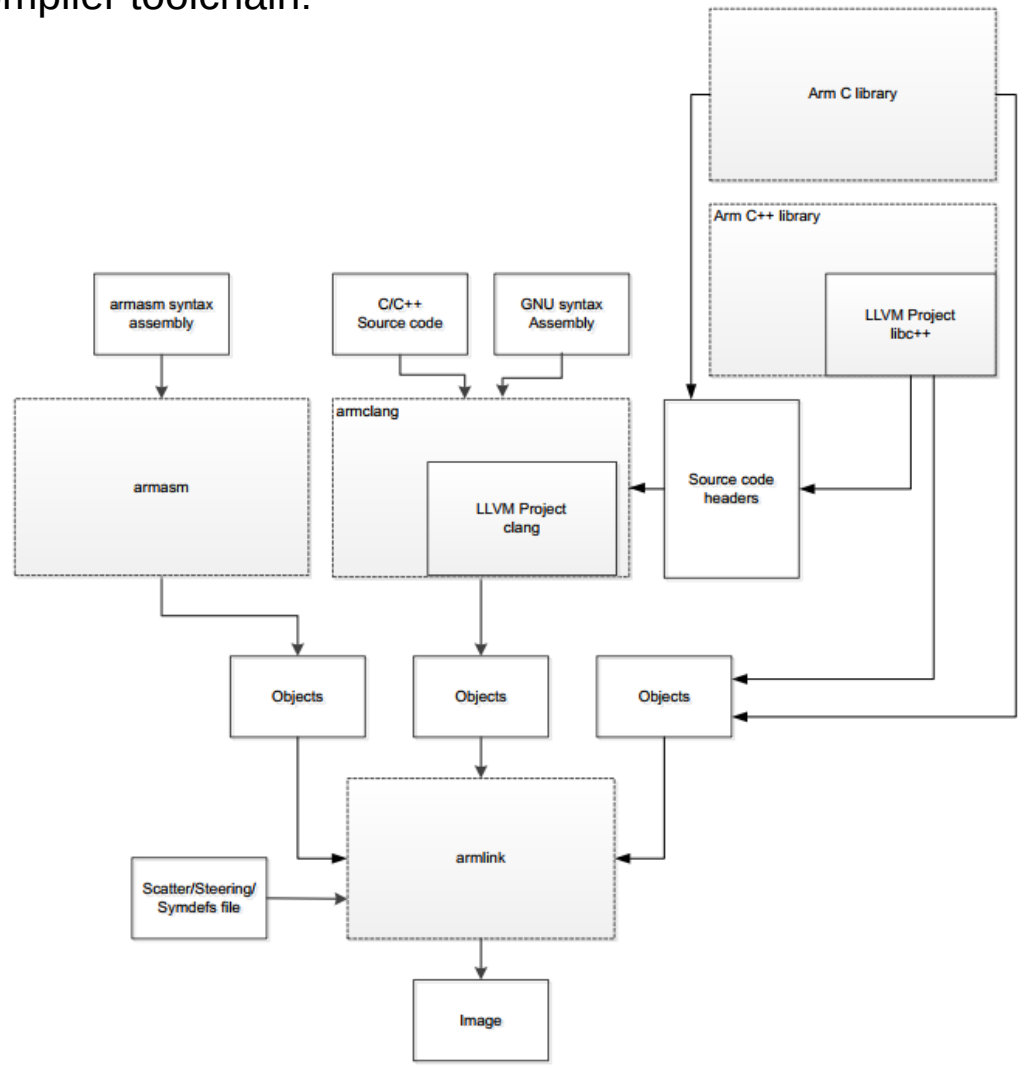
1. Overview Compiler and Arm Compiler

Arm Compiler combines the optimized tools and libraries from Arm with **a modern LLVM-based compiler framework**. The components in Arm Compiler 6 are:

- **armclang:** The compiler and integrated assembler that compiles **C**, **C++**, and **GNU assembly language** sources.
- **armasm:** The legacy assembler. Only use armasm for **legacy Arm-syntax assembly code**. Use the armclang assembler and GNU syntax for all new assembly files.
- **armlink:** The linker combines the contents of one or more **object files** with selected parts of one or more object libraries to produce an **executable program**.
- **armar:** The archiver enables sets of **ELF object files** to be collected together and maintained in archives or libraries.
- **fromelf:** The image conversion utility can convert **Arm ELF images** to **binary format**s. It can also generate textual information about the input image, such as its disassembly, code size, and data size.
- **Arm C++ libraries:** The Arm C++ libraries are based on the LLVM libc++ project.
- **Arm C libraries:** The Arm C libraries.

1. Overview Compiler and Arm Compiler

The following figure shows the **structure** of the Arm Compiler toolchain:

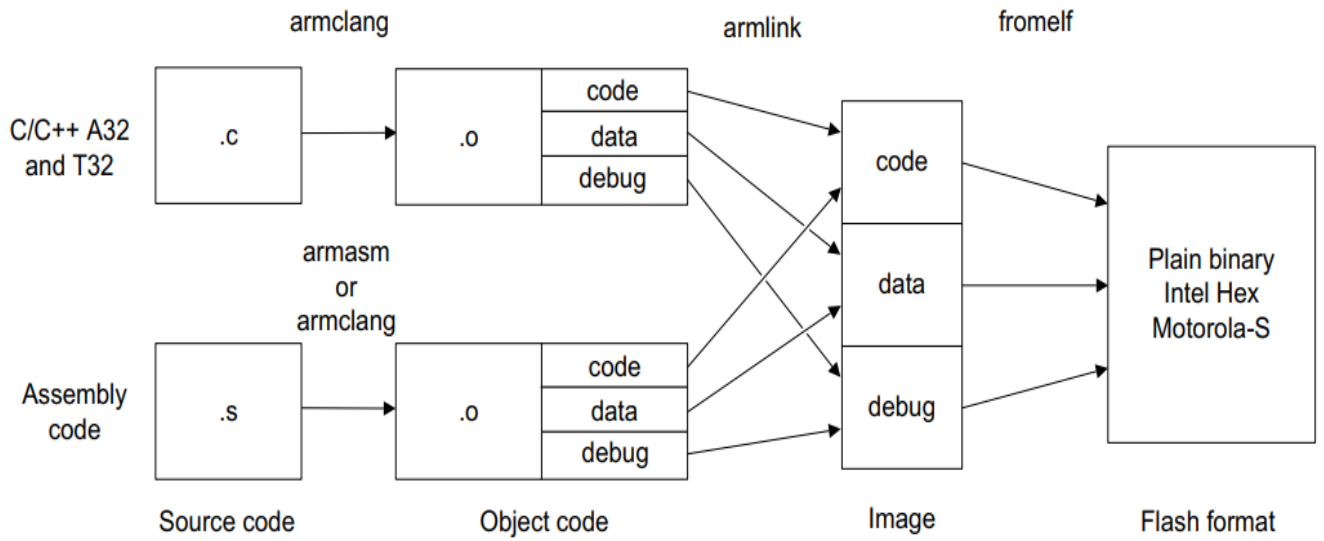


1. Overview Compiler and Arm Compiler

Application development**:** The following figure shows **how the compilation tools are used** for the development of a typical application.

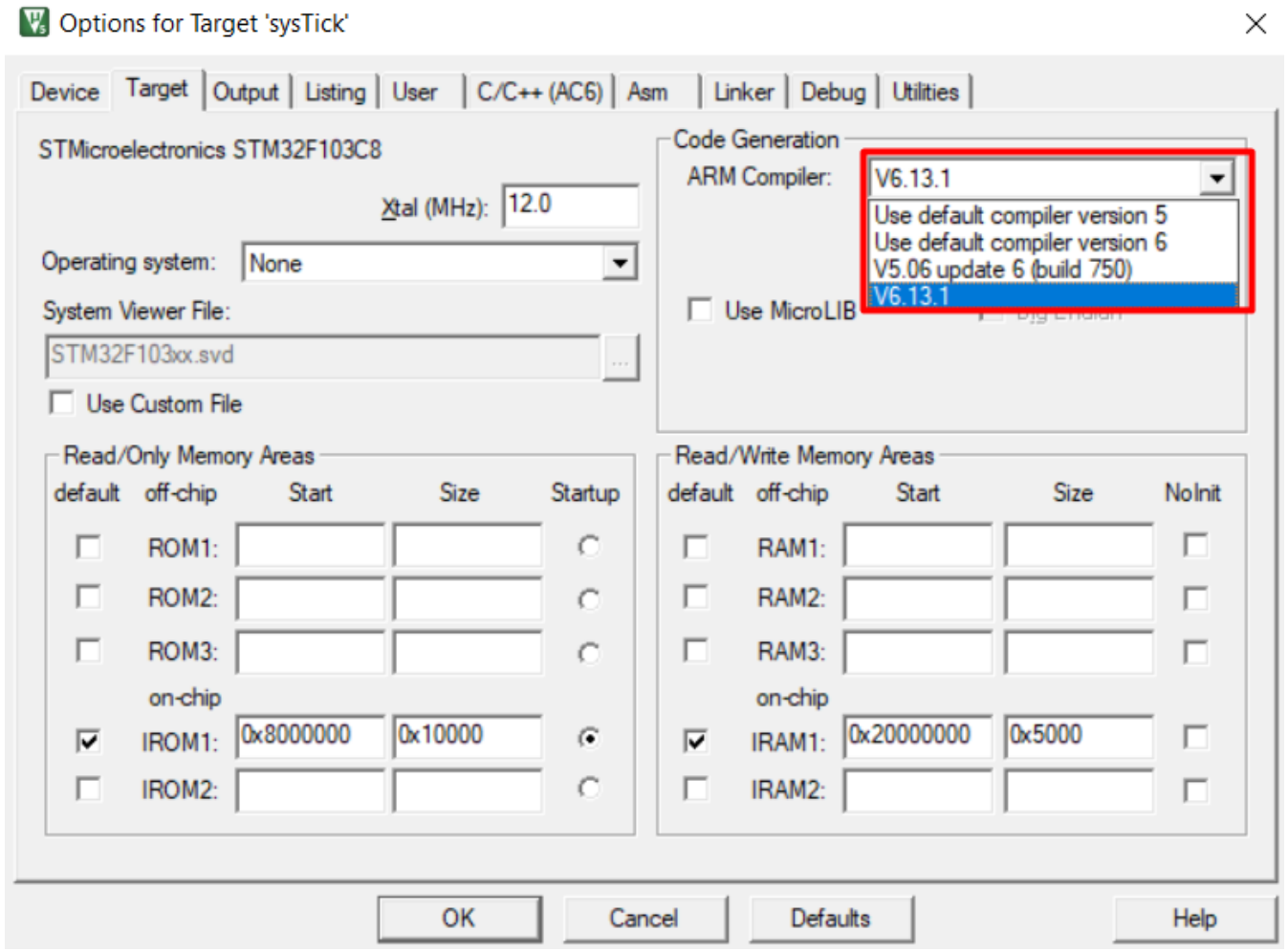
A typical application development flow might involve the following:

- ❑ Developing C/C++ source code for the main application (armclang).
- ❑ Developing assembly source code for near-hardware components, such as interrupt service routines (armclang, or armasm for legacy assembly code).
- ❑ Linking all objects together to generate an image (armlink).
- ❑ Converting an image to flash format in plain binary, Intel Hex, and Motorola-S formats (fromelf).



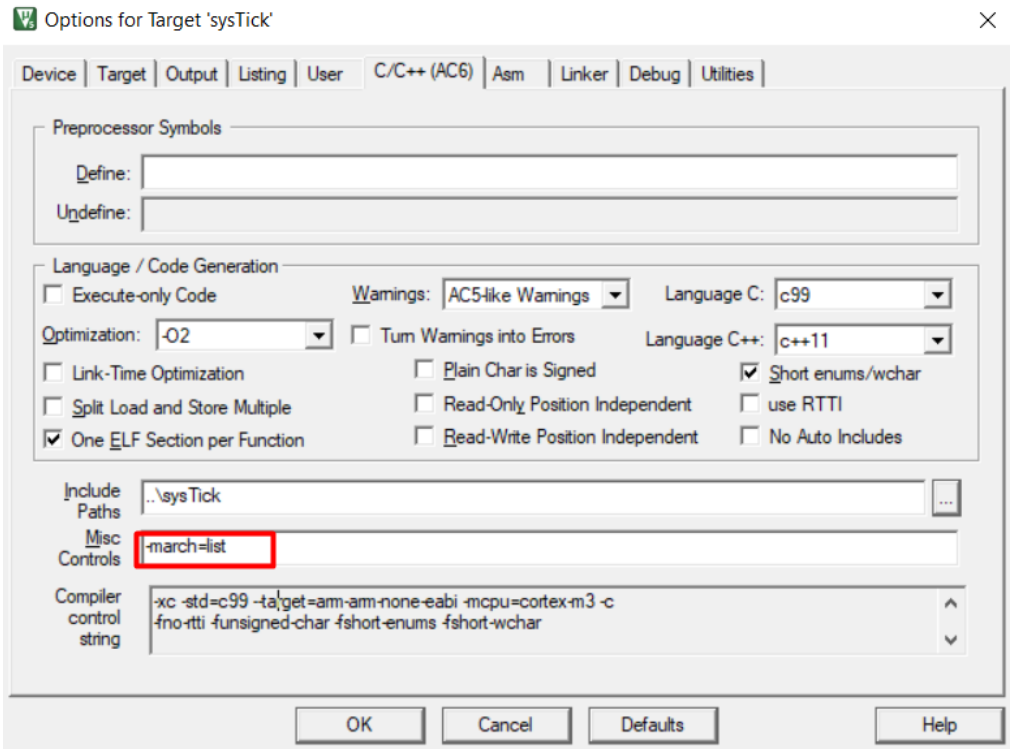
1. Overview Compiler and Arm Compiler

Using Arm compiler in KeilC IDE: MDK is a microprocessor development suite that provides the µVision® IDE, and Arm Compiler as a built-in toolchain.



1. Overview Compiler and Arm Compiler

- Using Arm compiler in KeilC IDE
  - User add compiler option



  - The output log after build the project

```
The following arguments to option 'march' can be selected:
  -march=armv6-m
  -march=armv7-a
  -march=armv7-m
  -march=armv7-r
  -march=armv7e-m
  -march=armv8-m.base
  -march=armv8-m.main
```

1. Overview Compiler and Arm Compiler

- Using Arm compiler directly by using command line

```
PC@DESKTOP-QEN4LJI ~
$ cd "C:\Keil_v5\ARM\ARMCLANG\bin"

PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ls
AnalyticsDll.dll                          api-ms-win-core-rtlsupport-l1-1-0.dll  api-ms-win-crt-string-l1-1-0.dll
api-ms-win-core-console-l1-1-0.dll        api-ms-win-core-string-l1-1-0.dll      api-ms-win-crt-time-l1-1-0.dll
api-ms-win-core-datetime-l1-1-0.dll       api-ms-win-core-synch-l1-1-0.dll       api-ms-win-crt-utility-l1-1-0.dll
api-ms-win-core-debug-l1-1-0.dll          api-ms-win-core-synch-l1-2-0.dll       armar.exe
api-ms-win-core-errorhandling-l1-1-0.dll  api-ms-win-core-sysinfo-l1-1-0.dll     armasm.exe
api-ms-win-core-file-l1-1-0.dll           api-ms-win-core-timezone-l1-1-0.dll    armclang.exe
api-ms-win-core-file-l1-2-0.dll           api-ms-win-core-util-l1-1-0.dll        armcompiler_libFNP.dll
api-ms-win-core-file-l2-1-0.dll           api-ms-win-crt-conio-l1-1-0.dll        armlink.exe
api-ms-win-core-handle-l1-1-0.dll         api-ms-win-crt-convert-l1-1-0.dll      concrt140.dll
api-ms-win-core-heap-l1-1-0.dll           api-ms-win-crt-environment-l1-1-0.dll  fromelf.exe
api-ms-win-core-interlocked-l1-1-0.dll    api-ms-win-crt-filesystem-l1-1-0.dll   libclang.dll
api-ms-win-core-libraryloader-l1-1-0.dll  api-ms-win-crt-heap-l1-1-0.dll         LTO.dll
api-ms-win-core-localization-l1-2-0.dll   api-ms-win-crt-locale-l1-1-0.dll       msvcp140.dll
api-ms-win-core-memory-l1-1-0.dll         api-ms-win-crt-math-l1-1-0.dll         ruiSDK_5.2.0.x86.dll
api-ms-win-core-namedpipe-l1-1-0.dll      api-ms-win-crt-multibyte-l1-1-0.dll    ucrtbase.dll
api-ms-win-core-processenvironment-l1-1-0.dll  api-ms-win-crt-private-l1-1-0.dll  UvClang.dll
api-ms-win-core-processthreads-l1-1-0.dll  api-ms-win-crt-process-l1-1-0.dll     vccorlib140.dll
api-ms-win-core-processthreads-l1-1-1.dll  api-ms-win-crt-runtime-l1-1-0.dll     vcruntime140.dll
api-ms-win-core-profile-l1-1-0.dll         api-ms-win-crt-stdio-l1-1-0.dll
```

1. Overview Compiler and Arm Compiler

armclang

- Using Arm compiler directly by using command line
- Note: the detail information about the compiler option is described in the later chapter.
- Example: Get the version of armclang, using the following command

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armclang -v
Product: MDK Plus 5.29
Component: ARM Compiler 6.13.1
Tool: armclang [5d895d00]

Target: unspecified-arm-none-unspecified
armclang: fatal error: no target architecture given; use --target=arm-arm-none-eabi or --target=aarch64-arm-none-eabi
```

- To see a list of all the supported architectures for the selected target

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armclang --target=arm-arm-none-eabi -march=list
The following arguments to option 'march' can be selected:
  -march=armv6-m
  -march=armv7-a
  -march=armv7-m
  -march=armv7-r
  -march=armv7e-m
  -march=armv8-m.base
  -march=armv8-m.main
```

1. Overview Compiler and Arm Compiler

Armclang

→ Get help on command line with Armclang

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armclang --help
Product: MDK Plus 5.29
Component: ARM Compiler 6.13.1
Tool: armclang [5d895d00]

Target: unspecified-arm-none-unspecified

Usage: armclang --target=<triple> [options] file1 file2 ... fileN

Mandatory options:
--target        Generate code for the specified target triple. Supported values are:
                  "arm-arm-none-eabi"     - generate code for AArch32 state
                  "aarch64-arm-none-eabi" - generate code for AArch64 state

Compiler and integrated assembler options:
-march          Targets an architecture, generating generic code that runs on any
                processor of that architecture.
-mcpu           Targets a specific processor, generating optimized code for that
                specific processor.
-mfpu           Targets a specific floating-point unit.
-march=list
-mcpu=list
-mfpu=list      Show a list of available architectures, CPUs and FPUs.
-mbig-endian    Generate code for a big-endian processor.
-mlittle-endian Generate code for a little-endian processor
```

1. Overview Compiler and Arm Compiler

Armlink

- Get help on command line with Armlink

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armlink --help
Product: MDK Plus 5.29
Component: ARM Compiler 6.13.1
Tool: armlink [5d895400]
For support see http://www.arm.com/support
Software supplied by: ARM Limited

Usage: armlink option-list input-file-list
where
    option-list      is a list of case-insensitive options.
    input-file-list  is a list of input object and library files.

General options (abbreviations shown capitalised):
   --help           Print this summary.
   --output file    Specify the name of the output file.
   --via file       Read further arguments from file.

Options for specifying memory map information:
   --partial        Generate a partially linked object.
   --scatter file   Create the memory map as described in file.
   --ro-base n      Set exec addr of region containing RO sections.
```

1. Overview Compiler and Arm Compiler

fromelf

- Get help on command line with **fromelf**

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./fromelf --help
Product: MDK Plus 5.29
Component: ARM Compiler 6.13.1
Tool: fromelf [5d895400]
For support see http://www.arm.com/support
Software supplied by: ARM Limited

ARM image conversion utility
fromelf [options] input_file

Options:
       --help          display this help screen
       --vsn           display version information
       --output file   the output file. (defaults to stdout for -text format)
       --nodebug       do not put debug areas in the output image
       --nolinkview    do not put sections in the output image

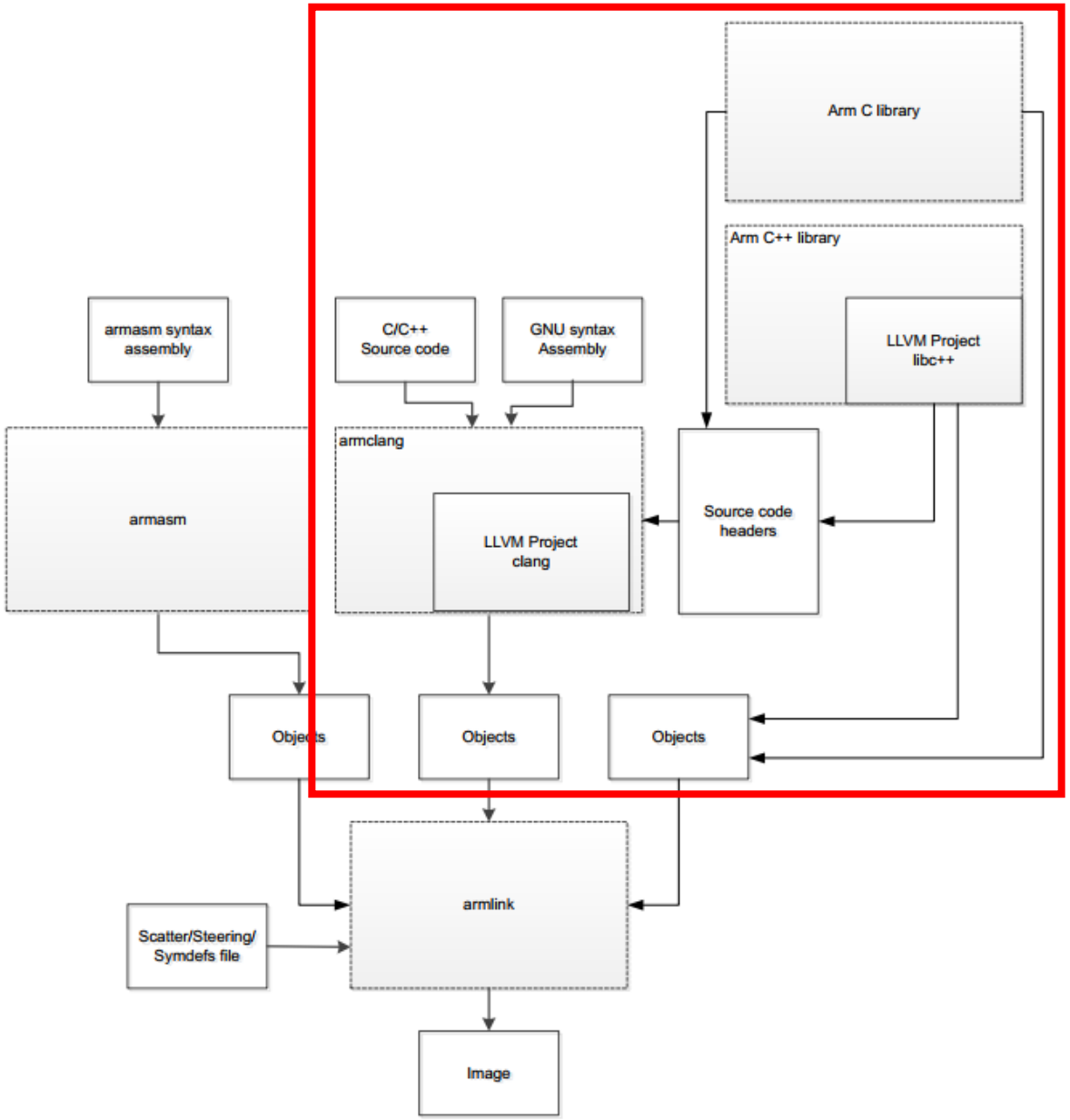
Binary Output Formats:
       --bin           Plain Binary
       --m32           Motorola 32 bit Hex
       --i32           Intel 32 bit Hex
       --vhx           Byte Oriented Hex format
```

Arm Compiler supported architectures

No	Architecture
1	Armv9-A
2	Armv8-A and all update releases, for bare-metal targets
3	Armv8-R
4	Armv8-M
5	Armv7-A for bare-metal targets
6	Armv7-R
7	Armv7-M
8	Armv6-M

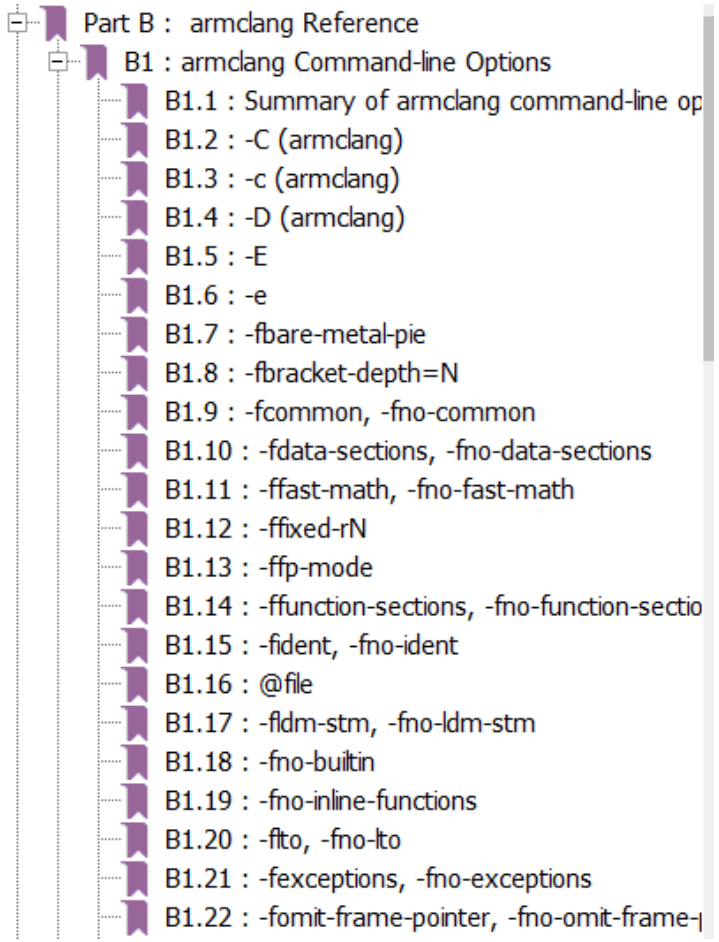
--target=<arch>-<vendor>-<os>-<abi>

2. Armclang

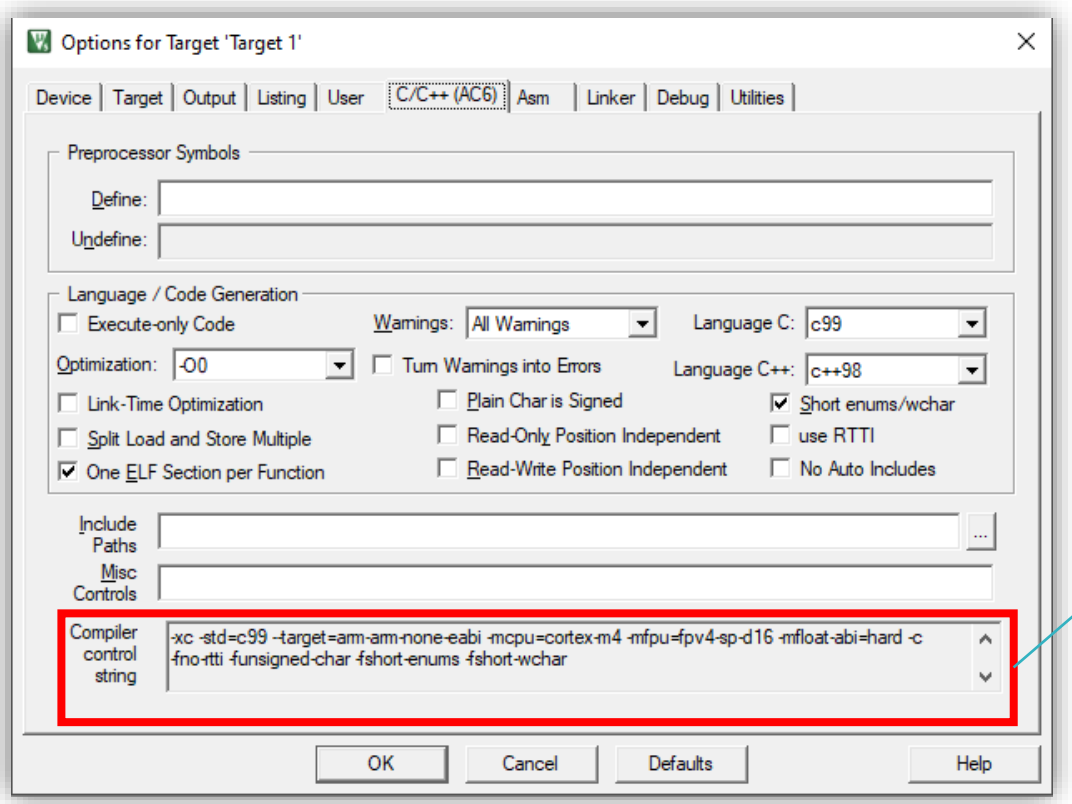

Arm C library
Arm C++ library
LLVM Project
libc++
armasm syntax
assembly
C/C++
Source code
GNU syntax
Assembly
armclang
armasm
LLVM Project
clang
Source code
headers
Objects
Objects
Objects
armlink
Scatter/Steering/
Symdefs file
Image

2. Armclang – Compiler option

- There are many options that you can use to control how Arm Compiler generates code for your application. This section lists the mandatory and commonly used optional command-line arguments, such as to control target selection, optimization, and debug view.



2. Armclang – Compiler option




```
-xc
-std=c99
--target=arm-arm-none-eabi
-mcpu=cortex-m4
-mfpu=fpv4-sp-d16
-mfloat-abi=hard
-c
-fno-rtti
-funsigned-char
-fshort-enums
-fshort-wchar
```

2. Armclang – Mandatory armclang options

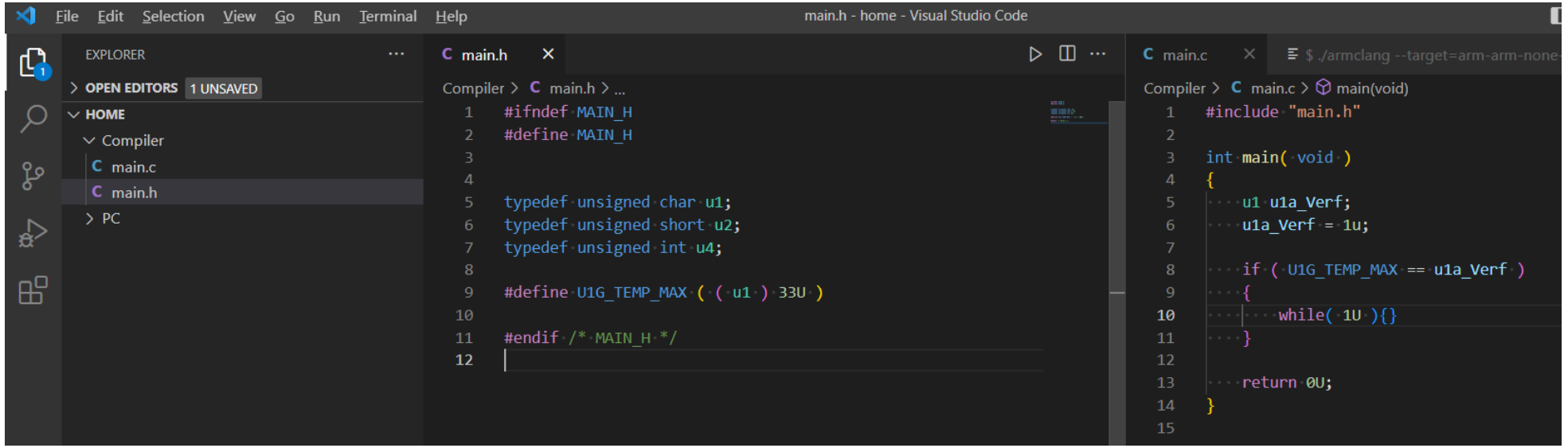
- When using armclang, you must specify a **target** on the command-line. Depending on the target you use, you might also have to specify an **architecture** or **processor**.
- To **specify a target**, use the *--target* option
  - To generate A64 instructions for AArch64 state, specify --target=aarch64-arm-none-eabi.
  - To generate A32 and T32 instructions for AArch32 state, specify --target=arm-arm-none-eabi. To specify generation of either A32 or T32 instructions, use *-marm* or *-mthumb* respectively.
- **Specifying an architecture**
  - To generate code for a specific architecture, use the *-march* option
  - To see a list of all the supported architectures for the selected target, use -march=list
- To generate code for **a specific processor**, use the *-mcpu* option
- To see a list of all the supported processors for the selected target, use -mcpu=list.
- If you want to run code on one particular processor, specify the processor using -mcpu. Performance is optimized, but code is only guaranteed to run on that processor.
- If you want your code to run on a range of processors from a particular architecture, specify the architecture using -march. The code runs on any processor implementation of the target architecture, but performance might be impacted

2. Armclang – Common Armclang options

- The common command line option:
  -c: Performs the compilation step, but not the link step.
  -x: Specifies the language of the subsequent source files, -xc inputfile.s or -xc++ inputfile.s for example.
  -std: Specifies the language standard to compile for, -std=c90 for example.
  -marm: Requests that the compiler targets the A32 instruction set, which consists of 32-bit wide instructions only. For example, --target=arm-arm-none-eabi -march=armv7-a -marm.
  -mthumb: Requests that the compiler targets the T32 instruction set, which consists of both 16-bit wide and 32-bit wide instructions. For example, --target=arm-arm-none-eabi -march=armv8-a -mthumb.
  -g: Generates DWARF debug tables compatible with the DWARF 4 standard.
  -E: Executes only the preprocessor step.
  -I: Adds the specified directories to the list of places that are searched to find included files.
  -o: Specifies the name of the output file.
  -Onum: Specifies the level of performance optimization to use when compiling source files.
  -Os: Balances code size against code speed.
  -Oz: Optimizes for code size.
  -S: Outputs the disassembly of the machine code that the compiler generates.
  -### Displays diagnostic output showing the options that would be used to invoke the compiler and linker. The compilation and link steps are not performed
- Note: To see more compiler options, refer to the Arm compiler reference guide.

2. Armclang – Example

- As we know, the compilation process comprises the following step :
  - Step 1: Preprocessing
  - Step 2: Compilation
  - Step 3: Assemble
  - Step 4: Linking
- The example will create two files main.h and main.c, then using the Armclang do from step 1 to step 3. Step 4 is done in the later chapter.
- Note: the direction of these files can change according to the user's definition.
- The sample source is shown below:

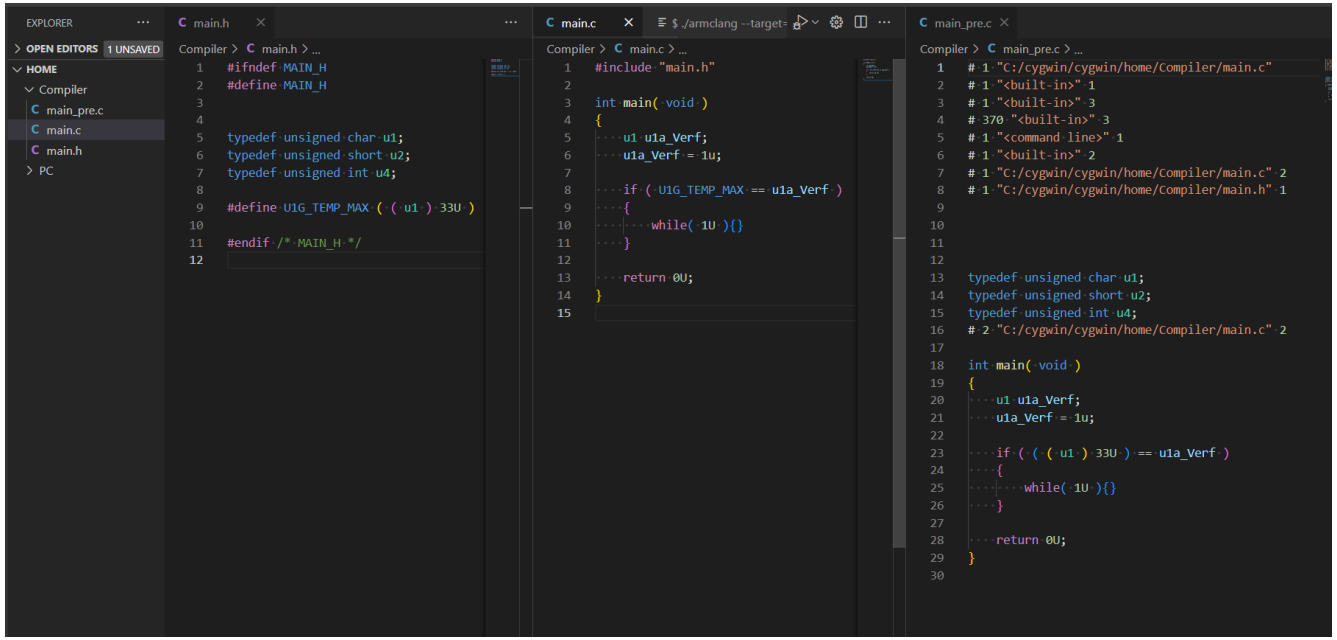


2. Armclang – Example

- Step 1: Preprocessing
  - Using the command line as below to create the main_pre.c which includes the headers (#include) and expands the macros (#define).
- The command: $ ./armclang --target=arm-arm-none-eabi -mcpu=cortex-m4 –E IC:/cygwin/cygwin/home/Compiler/main.h C:/cygwin/cygwin/home/Compiler/main.c -o C:/cygwin/cygwin/home/Compiler/main_pre.c

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armclang --target=arm-arm-none-eabi -mcpu=cortex-m4 -E -IC:/cygwin/cygwin/home/C
ompiler/main.h C:/cygwin/cygwin/home/Compiler/main.c -o C:/cygwin/cygwin/home/Compil
er/main_pre.c
```

- The result:

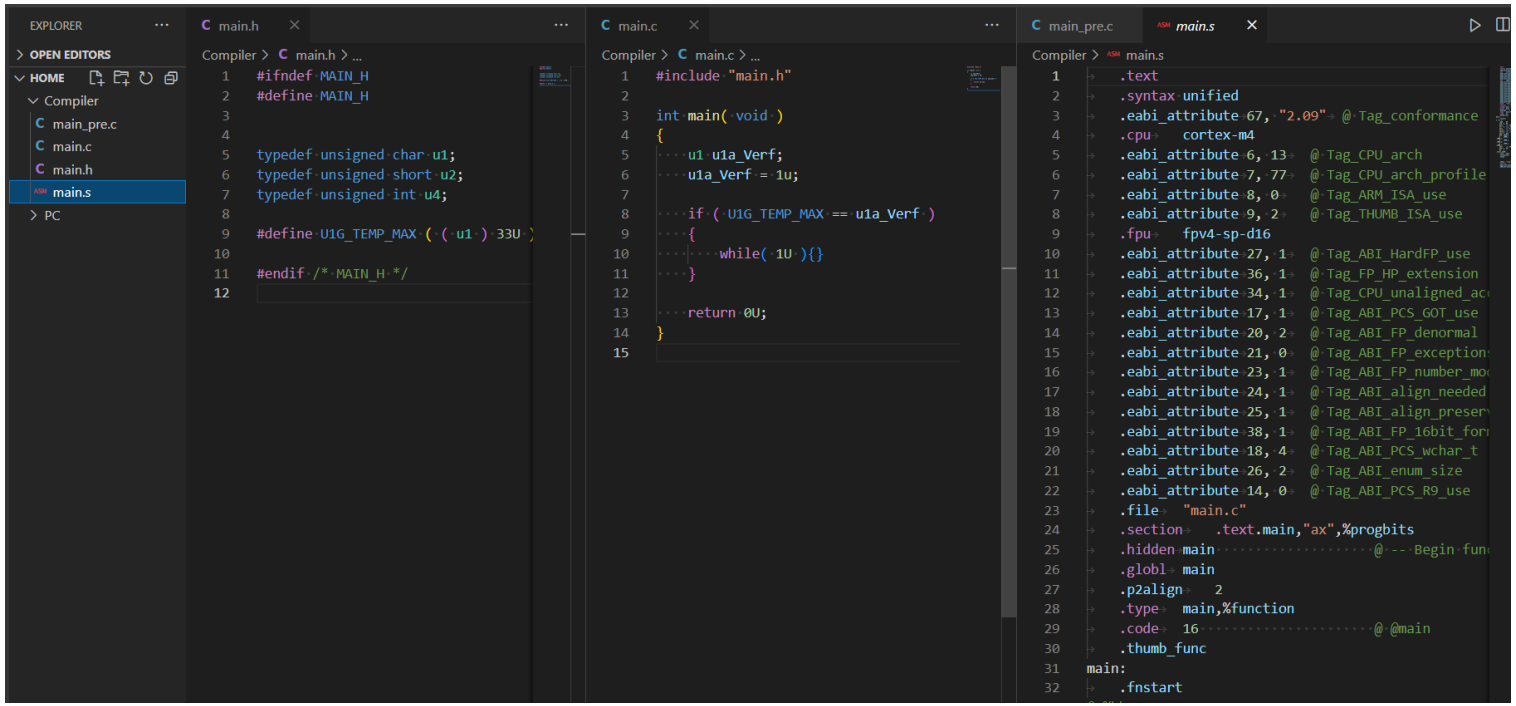


2. Armclang – Example

- Step 2: Compilation.
  - Using the command line as below to create the main.s
- Command: ./armclang --target=arm-arm-none-eabi -mcpu=cortex-m4 -S IC:/cygwin/cygwin/home/Compiler/main.h C:/cygwin/cygwin/home/Compiler/main.c -o C:/cygwin/cygwin/home/Compiler/main.s

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armclang --target=arm-arm-none-eabi -mcpu=cortex-m4 -S -IC:/cygwin/cygwin/home/C
ompiler/main.h C:/cygwin/cygwin/home/Compiler/main.c -o C:/cygwin/cygwin/home/Compil
er/main.s
```

- The result:

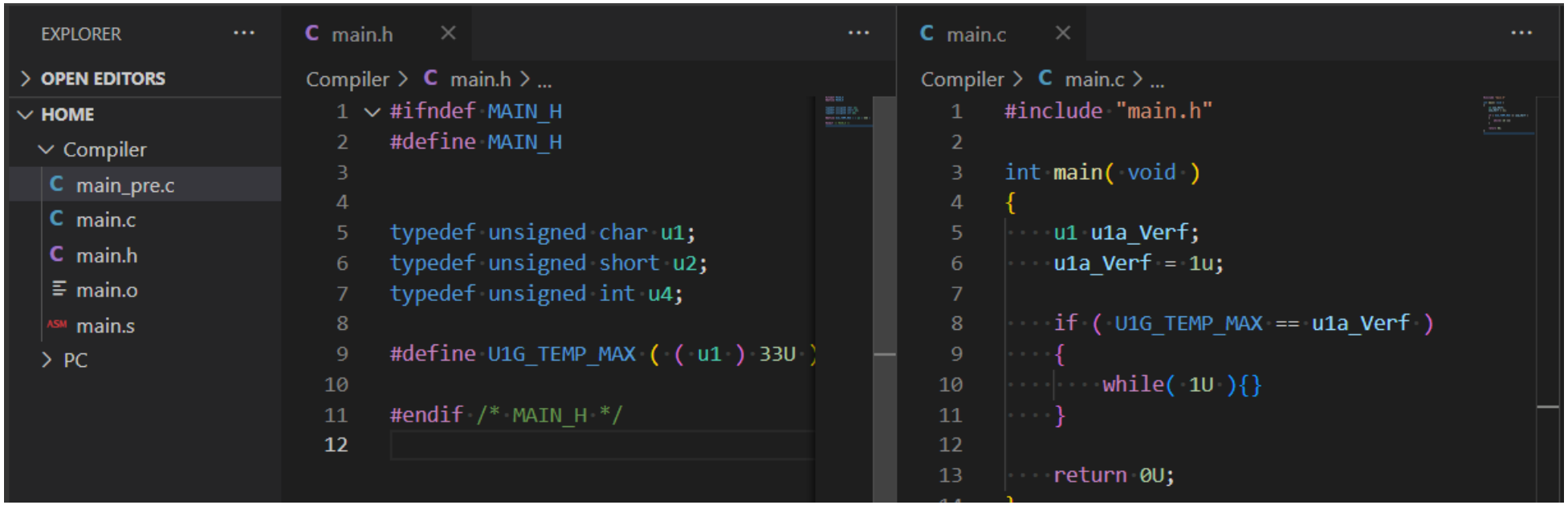


2. Armclang – Example

- Step 3: Assemble.
  - Using the command line as below to create the main.o
- Command: ./armclang --target=arm-arm-none-eabi -mcpu=cortex-m4 -c -IC:/cygwin/cygwin/home/Compiler/main.h C:/cygwin/cygwin/home/Compiler/main.c -o C:/cygwin/cygwin/home/Compiler/main.o

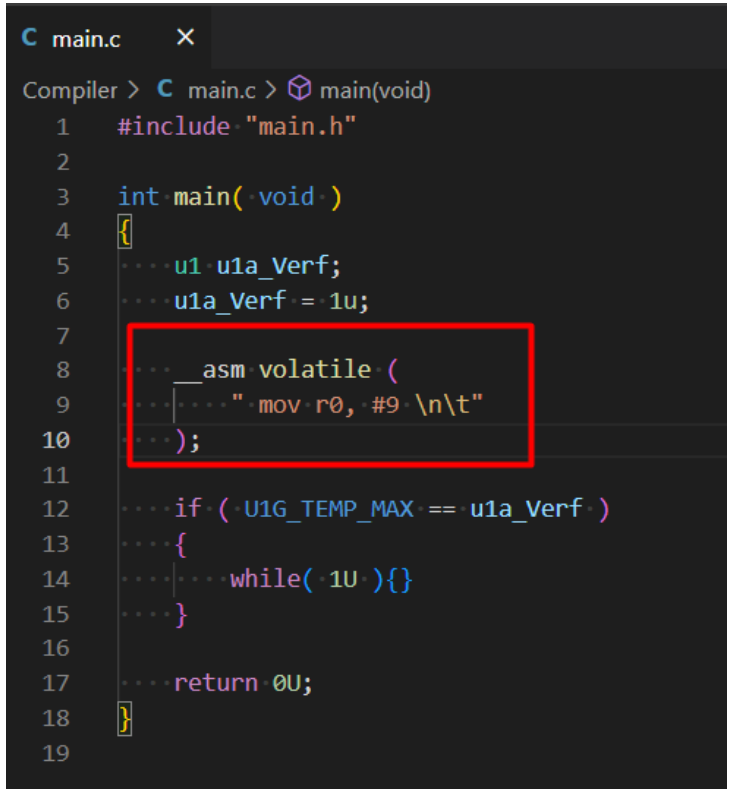
```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armclang --target=arm-arm-none-eabi -mcpu=cortex-m4 -c -IC:/cygwin/cygwin/home/C
ompiler/main.h C:/cygwin/cygwin/home/Compiler/main.c -o C:/cygwin/cygwin/home/Compil
er/main.o
```

- The result:



2. Armclang – Specific Keywords

- There are many keywords which support by Arm compiler. This section lists the commonly Specific keywords.
- **__asm** : This keyword passes information to the armclang assembler.
- Note: The using assembly code in C is described in the later chapter.
- Example:



2. Armclang – Specific Keywords

- **__inline__**

  Specify this keyword on a function definition or declaration as a hint to the compiler to favor inlining of the function. However, for each function call, the compiler still decides whether to inline the function. This is equivalent to **__inline**.

- **__attribute__((always_inline))**

  Specify this function attribute on a function definition or declaration to tell the compiler to always inline this function, with certain exceptions such as for recursive functions. This overrides the **-fno-inline-functions** option.

- **__attribute__((noinline))**

  Specify this function attribute on a function definition or declaration to tell the compiler to not inline the function. This is equivalent to **__declspec(noinline)**.

- **-fno-inline-functions**:

  This is a compiler command-line option. Specify this option to the compiler to disable inlining. This option overrides the **__inline__** hint.

main.c

Compiler > main.c > main(void)

```c
#include "main.h"

__inline__ static void f_App_Sum( u1 u1a_Arg0, u1 u1a_Arg1, u1* p2u1a_Sum );

int main( void )
{
    u1 u1a_Verf;
    u1a_Verf = 1u;

    __asm volatile (
        " mov r1, #9 \n\t"
        " mov r0, r1 \n\t"
    );

    f_App_Sum( 3U, 5U, &u1a_Verf );

    if ( U1G_TEMP_MAX == u1a_Verf )
    {
        while( 1U ){}
    }

    return 0U;
}

static void f_App_Sum( u1 u1a_Arg0, u1 u1a_Arg1, u1* p2u1a_Sum )
{
    u1 u1a_Sum;

    u1a_Sum = u1a_Arg0 + u1a_Arg1;
    *p2u1a_Sum = u1a_Sum;

    return;
}

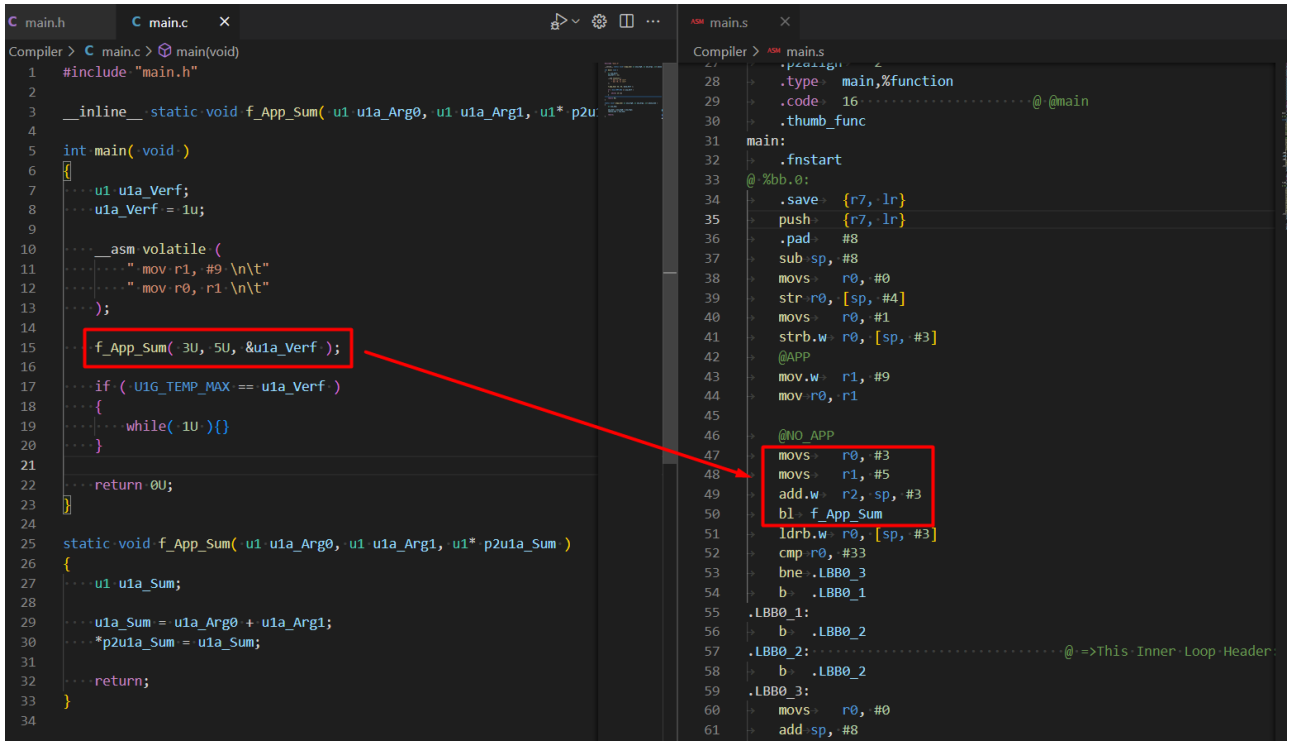
```

2. Armclang – Specific Attribute

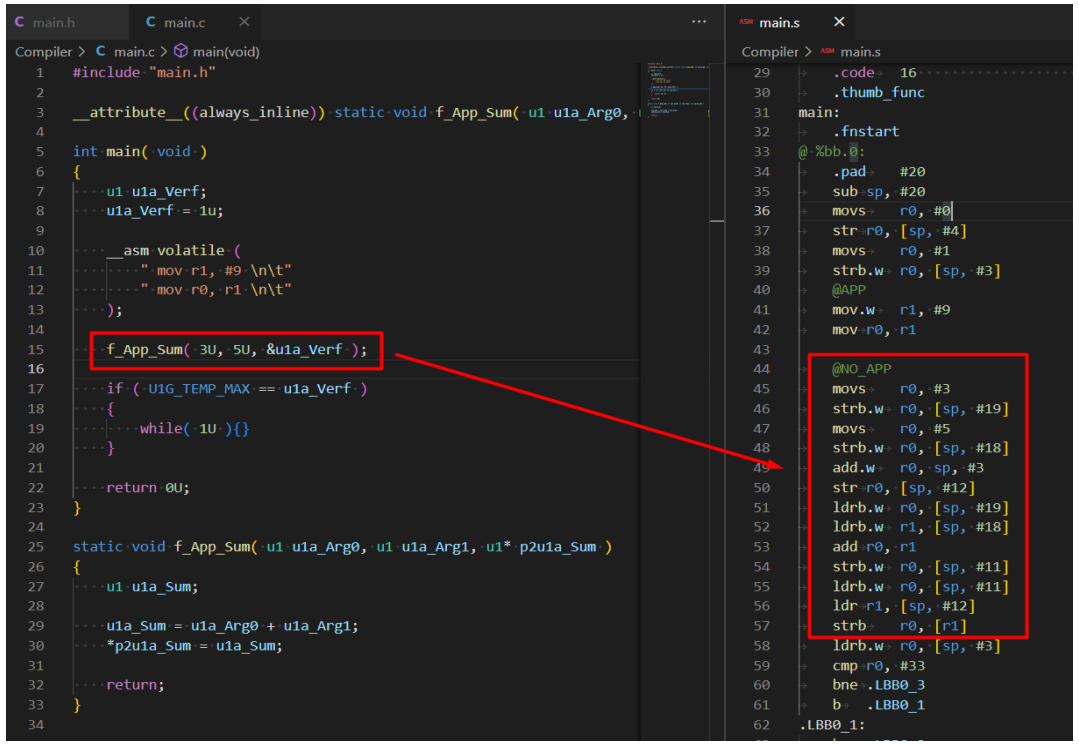
- Command to generate the assembly file (.s)

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armclang --target=arm-arm-none-eabi -mcpu=cortex-m4 -S -IC:/cygwin/cygwin/home/Compiler/main.h C:/cygwin/cygwin/hom
e/Compiler/main.c -o C:/cygwin/cygwin/home/Compiler/main.s
```

- Function f_App_Sum not inline by compiler



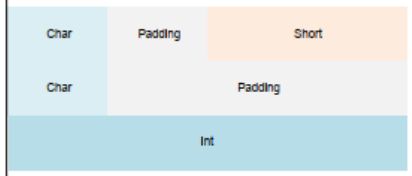
- Function f_App_Sum inline by compiler

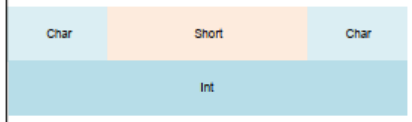


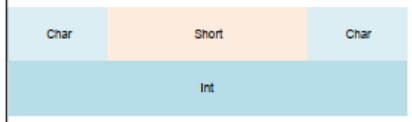
2. Armclang – Specific Pragma

- Packing data structures: #pragma pack (n). For each member, if n bytes is less than the natural alignment of the member, then set the alignment to n bytes, otherwise the alignment is the natural alignment of the member

Code	Packing	Size of structure

`struct stc`
`{`
`  char one;`
`  short two;`
`  char three;`
`  int four;`
`} c,d;`
 	
`int main (void)`
`{`
`  c.one=1;`
`  return 0;`
`}`
	
	Figure 4-1 Structure without packing attribute or pragma
		12. The alignment of the structure is the natural alignment of the largest member. In this example, the largest member is an `int`.

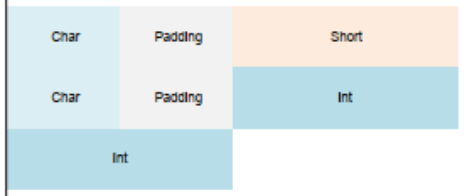
`struct __attribute__((packed))`
`stc`
`{`
`  char one;`
`  short two;`
`  char three;`
`  int four;`
`} c,d;`
 	
`int main (void)`
`{`
`  c.one=1;`
`  return 0;`
`}`
	
	Figure 4-2 Structure with attribute packed
		8. The alignment of the structure is 1 byte.

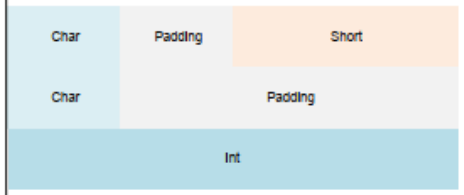
`#pragma pack (1)`
`struct stc`
`{`
`  char one;`
`  short two;`
`  char three;`
`  int four;`
`} c,d;`
 	
`int main (void)`
`{`
`  c.one=1;`
`  return 0;`
`}`
	
	Figure 4-3 Structure with pragma pack with 1 byte alignment
		8. The alignment of the structure is 1 byte.

2. Armclang – Specific Pragma

- Packing data structures: #pragma pack (n). For each member, if n bytes is less than the natural alignment of the member, then set the alignment to n bytes, otherwise the alignment is the natural alignment of the member

Code	Packing	Size of structure

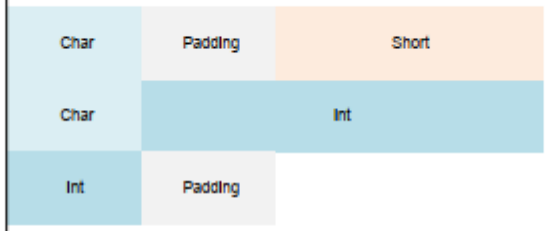
`#pragma pack (2)`
`struct stc`
`{`
`  char one;`
`  short two;`
`  char three;`
`  int four;`
`} c,d;`
 	
`int main (void)`
`{`
`  c.one=1;`
`  return 0;`
`}`
	 **Figure 4-4 Structure with pragma pack with 2 byte alignment**	10. The alignment of the structure is 2 bytes.

`#pragma pack (4)`
`struct stc`
`{`
`  char one;`
`  short two;`
`  char three;`
`  int four;`
`} c,d;`
 	
`int main (void)`
`{`
`  c.one=1;`
`  return 0;`
`}`
	 **Figure 4-5 Structure with pragma pack with 4 byte alignment**	12. The alignment of the structure is 4 bytes.

2. Armclang – Specific Attribute

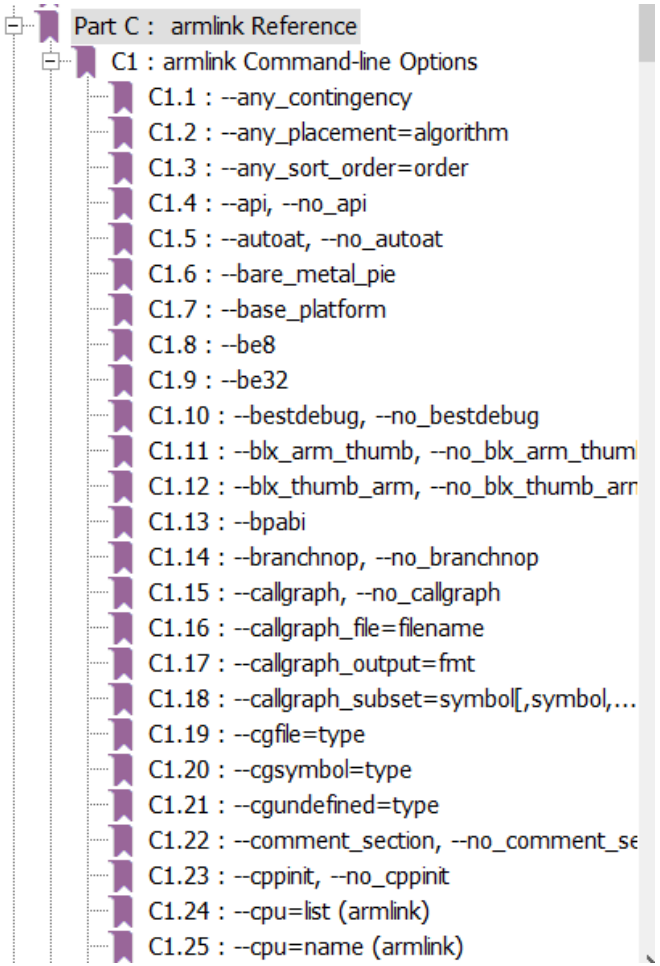
- Packing individual members in a structure: To pack individual members of a structure, use __attribute__((packed)) on the member. This aligns the member to a byte boundary and therefore reduces the amount of memory required by the structure as a whole. It does not affect the alignment of twhole structure is equal to the alignment of the largest member without the __attribute__((packed)).
  he other members. Therefore the alignment of the

Code	Packing	Size

`struct stc`
`{`
`  char one;`
`  short two;`
`  char three;`
`  int __attribute__((packed))`
`four;`
`} c,d;`
 	
`int main (void)`
`{`
`  c.one=1;`
`  return 0;`
`}`
	 **Figure 4-6 Structure with attribute packed on individual member**	10. The alignment of the structure is 2 bytes because the largest member without `__attribute__((packed))` is `short`.

2. Armlink - Overview

- There are many options that we can use to control the Armlink. This section lists the commonly used optional command-line in Armlink



2. Armlink - Command-line options

- **The Armlink command line option:**

  --cpu=name: Enables code generation for the selected Arm processor or architecture.

  --verbose: Displays detailed information about the link operation, including the objects that are included and the libraries that contain them.

  --info: Displays information about linker operation. For example, --info=sizes,unused,unusedsymbols displays information about all of the following:

    - Code and data sizes for each input object and library member in the image.
    - Unused sections that --remove has removed from the code.
    - Symbols that were removed with the unused sections.

  --list=filename: Redirects diagnostics output from options including --info and --map to the specified file.

  --map: Displays a memory map containing the address and the size of each load region, execution region, and input section in the image, including linker-generated input sections.

  --symbols: Lists each local and global symbol that is used in the link step, and their values.

  -o filename, **--output=filename**. Specifies the name of the output file.

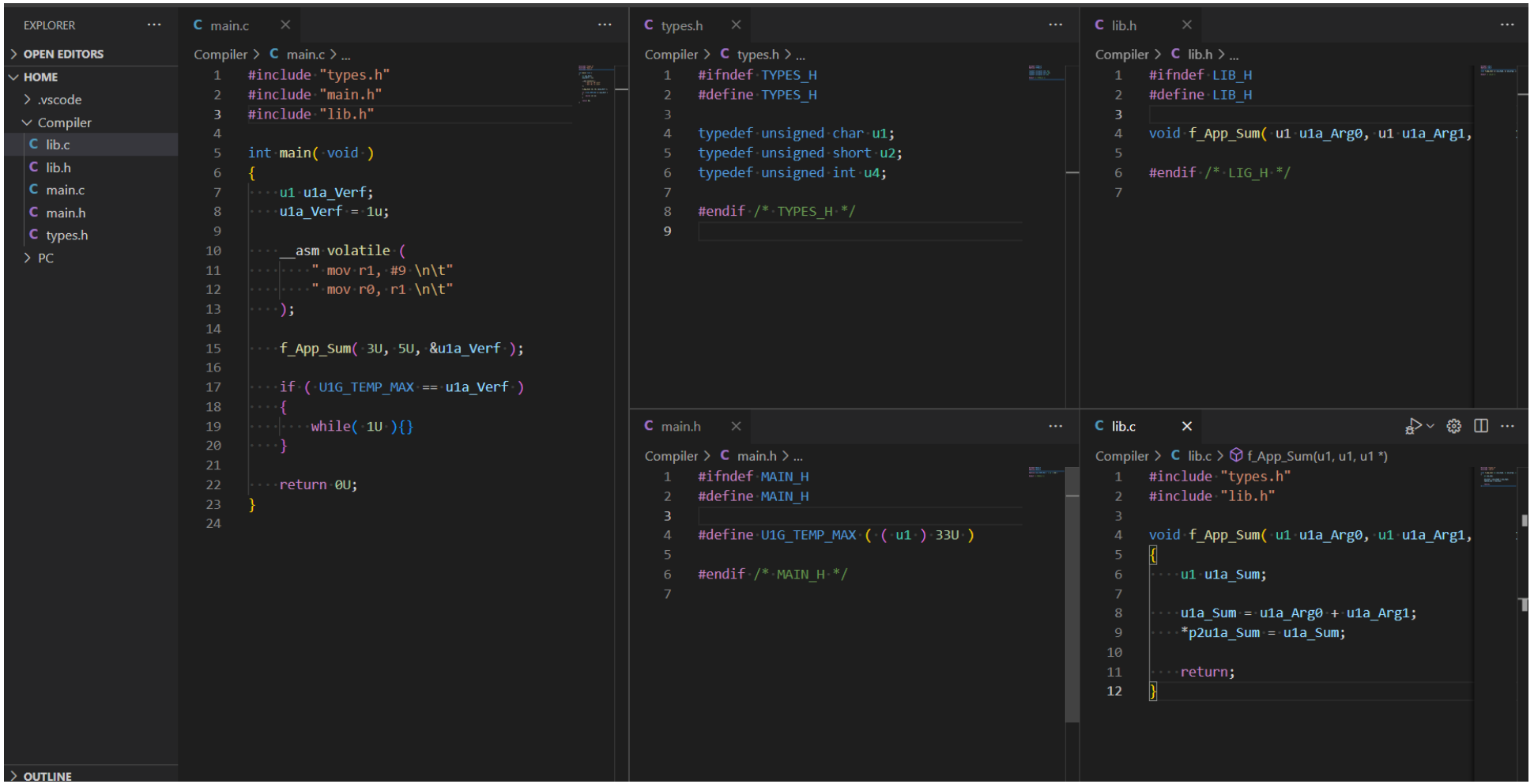
  --load_addr_map_info: Includes the load addresses for execution regions and the input sections within them in the map file.

  --entry: Specifies the unique initial entry point of the image.

  --scatter=filename: Creates an image memory map using the scatter-loading description that the specified file contains. This option relates to the linker file, so this option is described in the later chapter.

2. Armlink - Example

- Example: Using Armlink to link the obj file after compile by Armclang.
- Preparation: Create these folders as below:



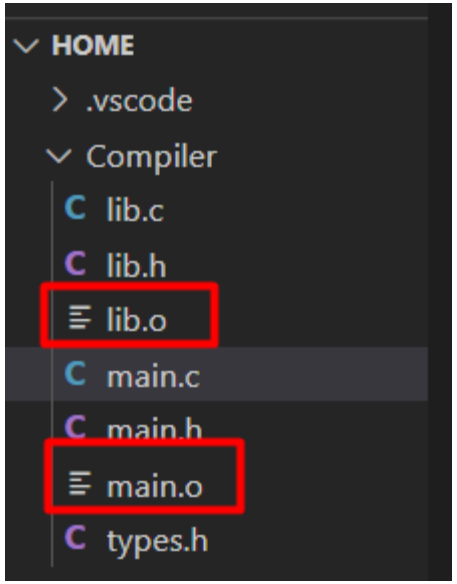
2. Armlink - Example

- Example: Using Armlink to link the obj file after compile by Armclang.
- Step 1: Using Armclang to create Obj file form the source file. (main.c, lib.c)
- The command line:

./armclang --target=arm-arm-none-eabi -mcpu=cortex-m4 -c -IC:/cygwin/cygwin/home/Compiler/main.h -IC:/cygwin/cygwin/home/Compiler/types.h -IC:/cygwin/cygwin/home/Compiler/lib.h C:/cygwin/cygwin/home/Compiler/main.c -o C:/cygwin/cygwin/home/Compiler/main.o
./armclang --target=arm-arm-none-eabi -mcpu=cortex-m4 -c -IC:/cygwin/cygwin/home/Compiler/main.h -IC:/cygwin/cygwin/home/Compiler/types.h -IC:/cygwin/cygwin/home/Compiler/lib.h C:/cygwin/cygwin/home/Compiler/lib.c -o C:/cygwin/cygwin/home/Compiler/lib.o

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armclang --target=arm-arm-none-eabi -mcpu=cortex-m4 -c -IC:/cygwin/cygwin/home/Compiler/main.h -IC:/cygwin/cygwin/home/Compiler/types.h -IC:/cygwin/cygwin/home/Compiler/lib.h C:/cygwin/cygwin/home/Compiler/lib.c -o C:/cygwin/cygwin/home/Compiler/lib.o

PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armclang --target=arm-arm-none-eabi -mcpu=cortex-m4 -c -IC:/cygwin/cygwin/home/Compiler/main.h -IC:/cygwin/cygwin/home/Compiler/types.h -IC:/cygwin/cygwin/home/Compiler/lib.h C:/cygwin/cygwin/home/Compiler/main.c -o C:/cygwin/cygwin/home/Compiler/main.o
```



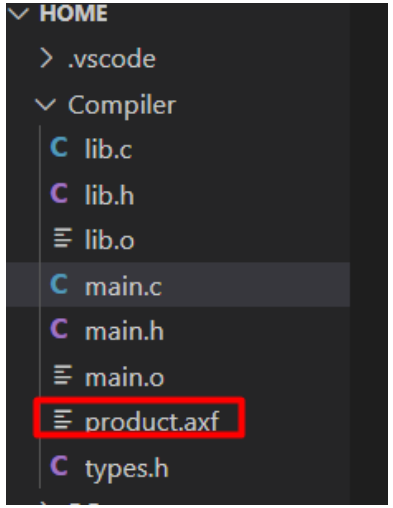
2. Armlink - Example

- Example: Using Armlink to link the obj file after compile by Armclang.
- Step 2: Using Armlink to produce executable images(product.axf)
- The command line:

./armlink --cpu=cortex-m4 C:/cygwin/cygwin/home/Compiler/main.o C:/cygwin/cygwin/home/Compiler/lib.o -o C:/cygwin/cygwin/home/Compiler/product.axf

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armlink --cpu=cortex-m4 C:/cygwin/cygwin/home/Compiler/main.o C:/cygwin/cygwin/home/Compiler/lib.o -o C:/cygwin/cyg
win/home/Compiler/product.axf
```

- The result:



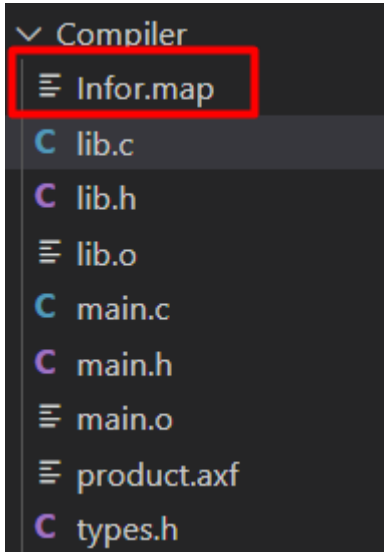
2. Armlink - Example

- Example: Using Armlink to link the obj file after compile by Armclang.
- Step 2: Using Armlink command-line option to get information from executable images(product.axf)
- The command:

./armlink --cpu=cortex-m4 --verbose --map --info=sizes,unused,unusedsymbols --symbols --load_addr_map_info --entry=main --list=C:/cygwin/cygwin/home/Compiler/Infor.map C:/cygwin/cygwin/home/Compiler/main.o C:/cygwin/cygwin/home/Compiler/lib.o -o C:/cygwin/cygwin/home/Compiler/product.axf

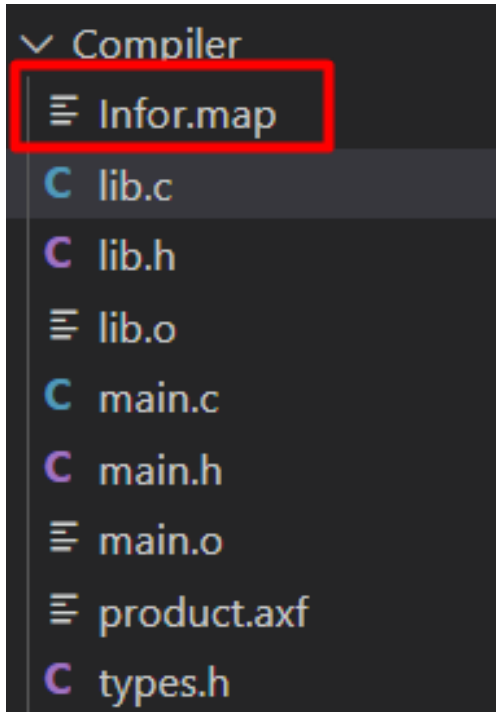
```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./armlink --cpu=cortex-m4 --verbose --map --info=sizes,unused,unusedsymbols --symbols --load_addr_map_info --entry=ma
in --list=C:/cygwin/cygwin/home/Compiler/Infor.map C:/cygwin/cygwin/home/Compiler/main.o C:/cygwin/cygwin/home/Compiler
/lib.o -o C:/cygwin/cygwin/home/Compiler/product.axf
```

- The result:



2. Armlink - Example

- Investigate the Infor.map



- With the --verbose option, the linking operation is shown below:

Infor.map

Compiler > Infor.map

```
Component: ARM Compiler 6.13.1 Tool: armlink [5d895400]

================================================================================

Loading Objects.

Loading object main.o.
            reference :  f_App_Sum
            definition:  main
Loading object lib.o.
            definition:  f_App_Sum
Adding startup reference.
            reference :  __main
Add decompression support symbols.
            reference :  __decompress_flags
            weak ref  :  __decompress_sizes
Adding attributes from cpu and fpu:

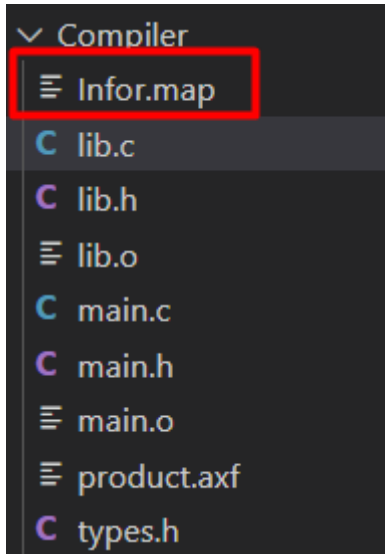
================================================================================

Loading System Libraries.

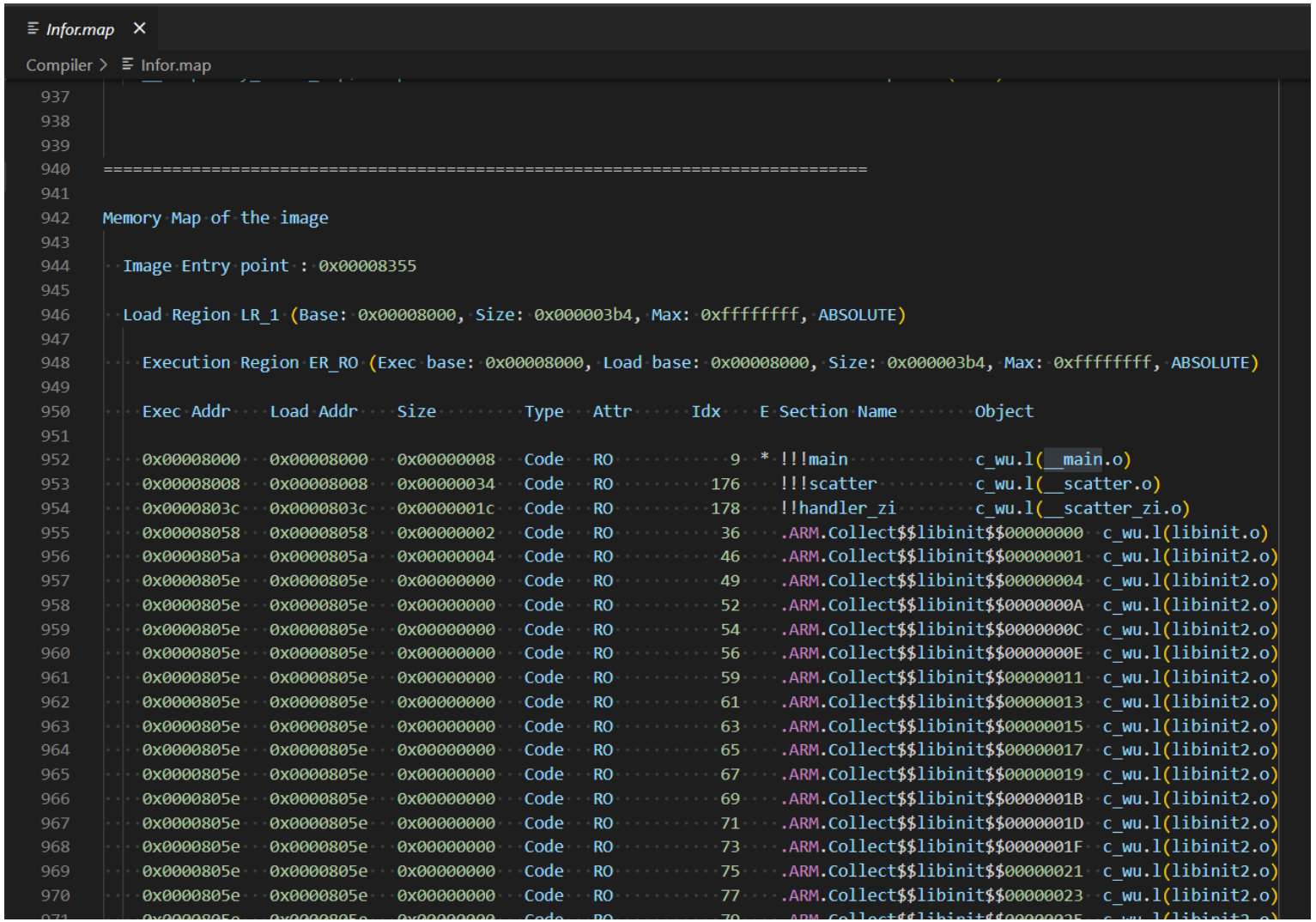
Searching for ARM libraries in directory C:\Keil_v5\ARM\ARMCLANG\bin\..\lib\armlib\
Searching for ARM libraries in directory C:\Keil_v5\ARM\ARMCLANG\bin\..\lib\libcxx\
Selecting library C:\Keil_v5\ARM\ARMCLANG\bin\..\lib\armlib\c_wu.l.
Selecting library C:\Keil_v5\ARM\ARMCLANG\bin\..\lib\armlib\fz_wm.l.
Selecting library C:\Keil_v5\ARM\ARMCLANG\bin\..\lib\armlib\h_wu.l.
Selecting library C:\Keil_v5\ARM\ARMCLANG\bin\..\lib\libcxx\libcpp-experimental_wnu.l.
Selecting library C:\Keil_v5\ARM\ARMCLANG\bin\..\lib\libcxx\libcpp_wnu.l.
Selecting library C:\Keil_v5\ARM\ARMCLANG\bin\..\lib\libcxx\libcppabi_wnu.l.
Selecting library C:\Keil_v5\ARM\ARMCLANG\bin\..\lib\armlib\m_wm.l.
Selecting library C:\Keil_v5\ARM\ARMCLANG\bin\..\lib\armlib\vfpsupport.l.
Selecting member __main.o(c_wu.l) to define __main.
Selecting member dc.o(c_wu.l) to define __decompress_flags.
Loading member dc.o from c_wu.l.
            definition:  __decompress_flags
```

2. Armlink - Example

- Investigate the Infor.map

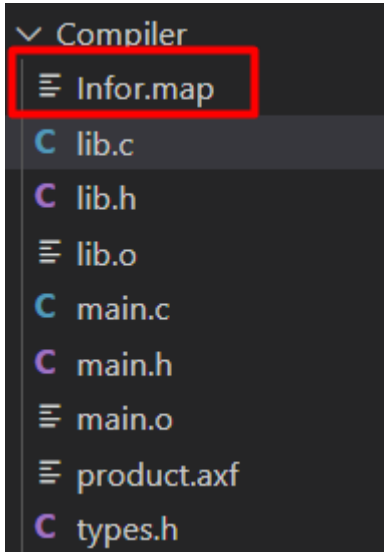


- With the --map option, the memory map is shown below



2. Armlink - Example

- Investigate the Infor.map



- With the --info=sizes,unused,unusedsymbols option, the unused,unusedsymbols information is shown below

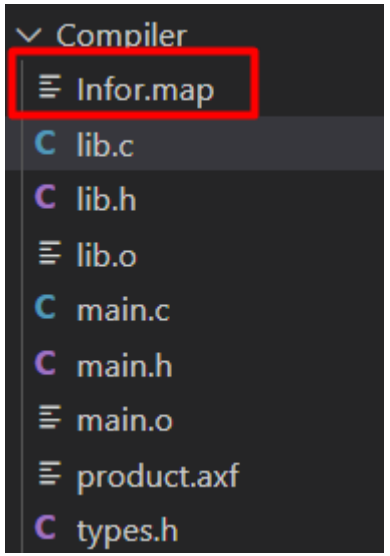
Infor.map

Compiler > Infor.map

```
663  Checking duplicate symbol definitions
664
665  ==============================================================================
666
667  Removing Unused input sections from the image.
668
669      Removing main.o(.text), (0 bytes).
670      Removing main.o(.ARM.exidx.text.main), (8 bytes).
671      Removing lib.o(.text), (0 bytes).
672      Removing lib.o(.ARM.exidx.text.f_App_Sum), (8 bytes).
673
674  4 unused section(s) (total 16 bytes) removed from the image.
675                  reference :  __scatterload_zeroinit
676                  reference :  __scatterload_null
677                  reference :  __scatterload_rt2
678  Choosing data area compressor
679  Data area too small to benefit from compression.
680  Selecting member __scatter_zi.o(c_wu.l) to define __scatterload_zeroinit.
681  Selecting member __scatter.o(c_wu.l) to define __scatterload_null.
682  Selecting member __scatter.o(c_wu.l) to define __scatterload_rt2.
683  Loading member __scatter.o from c_wu.l.
684                  definition:  __scatterload
685                  definition:  __scatterload_rt2
686                  definition:  __scatterload_rt2_thumb_only
687                  definition:  __scatterload_null
688                  reference :  Region$$Table$$Base
689                  reference :  Region$$Table$$Limit
690                  reference :  __rt_entry
691  Loading member __scatter_zi.o from c_wu.l.
692                  definition:  __scatterload_zeroinit
693  Patching/Reducing relocations
694  Creating output file C:/cygwin/cygwin/home/Compiler/product.axf.
695
696  ==============================================================================
```

2. Armlink - Example

- Investigate the Infor.map



- With the --info=sizes,unused,unusedsymbols option, the sizes imformation is shown below

Infor.map

Compiler > Infor.map

```
0x000083b4   -   0x00000060   Zero   RW   38   .bss

==============================================================================

Image component sizes


    Code (inc. data)   RO Data   RW Data   ZI Data   Debug   Object Name

        42          0         0         0         0       0   lib.o
        52          0         0         0         0       0   main.o

    ------------------------------------------------------------------------
        96          0        16         0         0       0   Object Totals
         0          0        16         0         0       0   (incl. Generated)
         2          0         0         0         0       0   (incl. Padding)

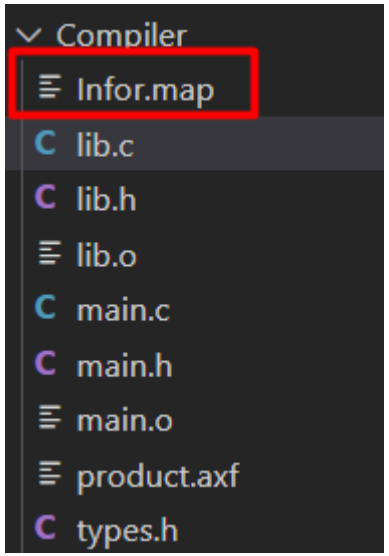
    ------------------------------------------------------------------------

    Code (inc. data)   RO Data   RW Data   ZI Data   Debug   Library Member Name

         8          0         0         0         0      68   __main.o
         0          0         0         0         0       0   __rtentry.o
        12          0         0         0         0       0   __rtentry2.o
         6          0         0         0         0       0   __rtentry4.o
        52          8         0         0         0       0   __scatter.o
        28          0         0         0         0       0   __scatter_zi.o
       302          0         0         0         0     168   _get_argv_nomalloc.o
         4          0         0         0         0      68   argv_veneer.o
        10          0         0         0         0      68   defsig_exit.o
        50          0         0         0         0      88   defsig_general.o
        80         58         0         0         0      76   defsig_rtmem_inner.o
        14          0         0         0         0      80   defsig_rtmem_outer.o
        18          0         0         0         0      80   exit.o
         4          0         0         0         0     136   hrguard.o
         0          0         0         0         0       0   indicate_semi.o
         2          0         0         0         0       0   libinit.o
```

2. Armlink - Example

- Investigate the Infor.map



- With the --symbols option, the local symbol information is shown below

```
Image Symbol Table

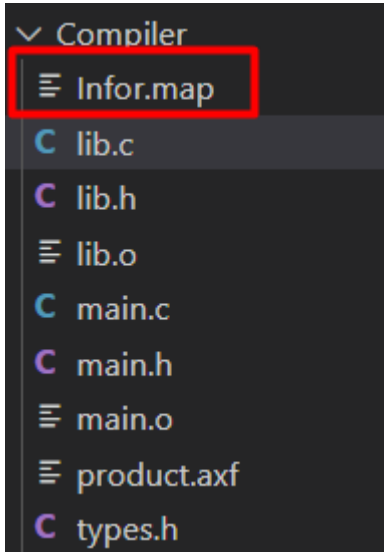
    Local Symbols

    Symbol Name                                Value     Ov Type        Size  Object(Section)

    ../clib/angel/boardlib.s                   0x00000000   Number         0  boardinit1.o ABSOLUTE
    ../clib/angel/boardlib.s                   0x00000000   Number         0  boardinit2.o ABSOLUTE
    ../clib/angel/boardlib.s                   0x00000000   Number         0  boardinit3.o ABSOLUTE
    ../clib/angel/boardlib.s                   0x00000000   Number         0  boardshut.o ABSOLUTE
    ../clib/angel/handlers.s                   0x00000000   Number         0  __scatter_zi.o ABSOLUTE
```

2. Armlink - Example

- Investigate the Infor.map



- With the --symbols option, the global symbol imformation is shown below

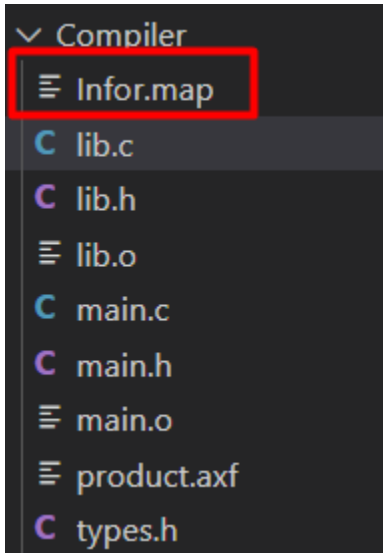
```
Global Symbols

Symbol Name                                Value    Ov Type        Size  Object(Section)

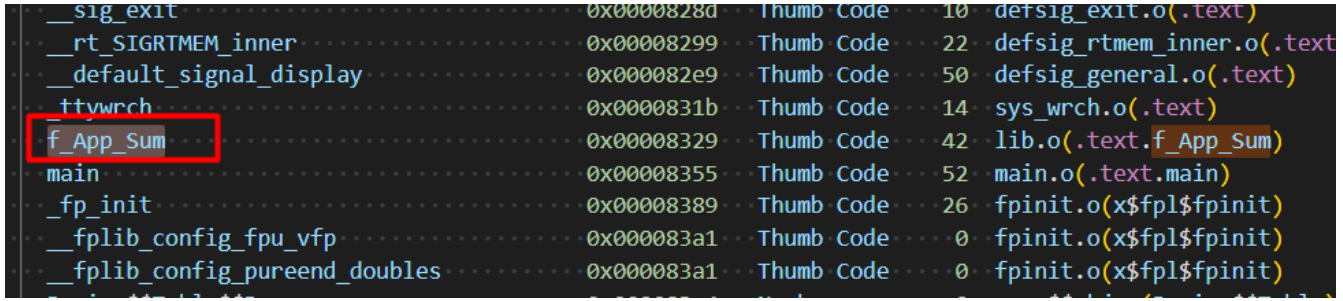
BuildAttributes$$THM_ISAv4$E$P$D$K$B$S$7EM$VFPi3$EXTD16$VFPS$VFMA$PE$A:L22UL41UL21$X:L11$S22
__ARM_exceptions_init                    - Undefined Weak Reference
__alloca_initialize                      - Undefined Weak Reference
__arm_preinit_                           - Undefined Weak Reference
__cpp_initialize__aeabi_                 - Undefined Weak Reference
__cxa_finalize                           - Undefined Weak Reference
__rt_locale                              - Undefined Weak Reference
```

2. Armlink - Example

- Investigate the Infor.map

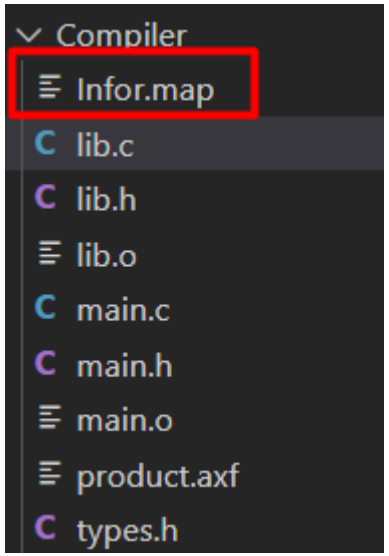


- With the --symbols option, the imformation is shown below
- Note: the symbols can be the variable or the function, we can see the f_App_Sum function in the global symbol section



2. Armlink - Example

- Investigate the Infor.map



- With the --load_addr_map_info option, the information is shown below
- Note: the “load addr” definition is described in the linker chapter.

```
==============================================================================

Memory Map of the image

  Image Entry point : 0x080000d1

  Load Region LR_IROM_1 (Base: 0x08000000, Size: 0x0000090c, Max: 0x00010000, ABSOLUTE)

    Execution Region LibRegion (Exec base: 0x08000000, Load base: 0x08000000, Size: 0x00000088, Max: 0xffffffff, ABSOLUTE)

    Exec Addr    Load Addr    Size         Type   Attr      Idx    E Section Name        Object

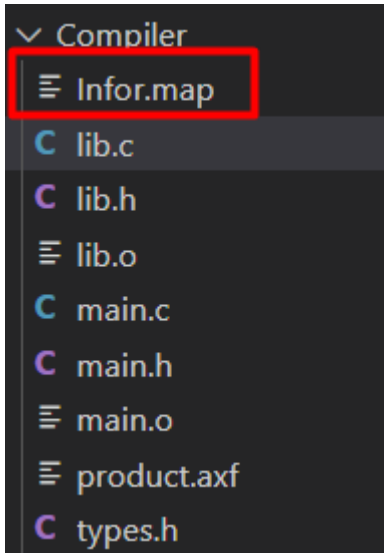
    0x08000000   0x08000000   0x00000008   Code   RO           40  * !!!main             c_wu.l(__main.o)
    0x08000008   0x08000008   0x00000034   Code   RO          212    !!!scatter          c_wu.l(__scatter.o)
    0x0800003c   0x0800003c   0x0000001a   Code   RO          214    !!handler_copy      c_wu.l(__scatter_copy.o)
    0x08000056   0x08000056   0x00000002   PAD
    0x08000058   0x08000058   0x00000030   Data   RO          211    Region$$Table       anon$$obj.o


    Execution Region ARM_LIB (Exec base: 0x08000088, Load base: 0x08000088, Size: 0x00000048, Max: 0xffffffff, ABSOLUTE)

    Exec Addr    Load Addr    Size         Type   Attr      Idx    E Section Name        Object
```

2. Armlink - Example

- Investigate the Infor.map



- With the --entry=main option, the information is shown below

```
    exit                                    0x080000b1   Thumb Code    18  exit.o(.text)
    __I$use$semihosting                     0x080000c3   Thumb Code     0  use_no_semi.o(.text)
    __use_no_semihosting_swi                0x080000c3   Thumb Code     2  use_no_semi.o(.text)
    __semihosting_library_function          0x080000c5   Thumb Code     0  indicate_semi.o(.text)
    _sys_exit                               0x080000c5   Thumb Code     8  sys_exit.o(.text)
    main                                    0x080000d1   Thumb Code   208  Sample.o(.text.main)
    vol_Helloworld_Print                    0x080001a1   Thumb Code    26  Helloworld.o(.text.vol_Helloworld_Print)
    u1l_en_print                            0x20000001   Data          1  Helloworld.o(.data.u1l_en_print)
    Image$$ARM_LIB_STACK$$ZI$$Limit         0x20005000   Number        0  anon$$obj.o ABSOLUTE


==============================================================================

Memory Map of the image

  Image Entry point : 0x080000d1

  Load Region LR_IROM_1 (Base: 0x08000000, Size: 0x0000090c, Max: 0x00010000, ABSOLUTE)

    Execution Region LibRegion (Exec base: 0x08000000, Load base: 0x08000000, Size: 0x00000088, Max: 0xffffffff, ABSOLUTE)
```

2. Fromelf - Overview

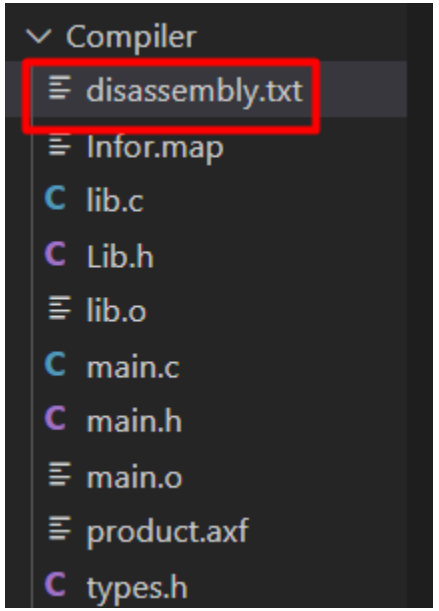
- The fromelf image conversion utility allows you to modify ELF image and object files, and to display information on those files.
- We can run fromelf in various execution modes.
- The execution modes are:
  - ELF mode (--elf), to resave a file as ELF.
  - Text mode (--text, and others), to output information about an object or image file.
  - Format conversion mode (--bin, --m32, --i32, --vhx).

2. Fromelf – Command-line option

- Fromelf command line option:
  - -c: disassemble code.
  - Command line:
    ./fromelf --text -c C:/cygwin/cygwin/home/Compiler/product.axf –o C:/cygwin/cygwin/home/Compiler/disassembly.txt

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./fromelf --text -c C:/cygwin/cygwin/home/Compiler/product.axf -o C:/cygwin/cygwin/home/Compiler/disassembly.txt
```

- The result:




```
    $t
    .text
    __argv_alloc
        0x00008134:    4602        .F      MOV     r2,r0
        0x00008136:    1cc9        ..      ADDS    r1,r1,#3
        0x00008138:    f0210103    !...    BIC     r1,r1,#3
        0x0000813c:    e9d20300    ....    LDRD    r0,r3,[r2,#0]
        0x00008140:    1a1b        ..      SUBS    r3,r3,r0
        0x00008142:    428b        .B      CMP     r3,r1
        0x00008144:    d201        ..      BCS     0x814a ; __argv_alloc + 22
        0x00008146:    2000        .       MOVS    r0,#0
        0x00008148:    4770        pG      BX      lr
        0x0000814a:    4401        .D      ADD     r1,r1,r0
        0x0000814c:    1dc9        ..      ADDS    r1,r1,#7
        0x0000814e:    f0210107    !...    BIC     r1,r1,#7
        0x00008152:    6011        .`      STR     r1,[r2,#0]
        0x00008154:    4770        pG      BX      lr
```

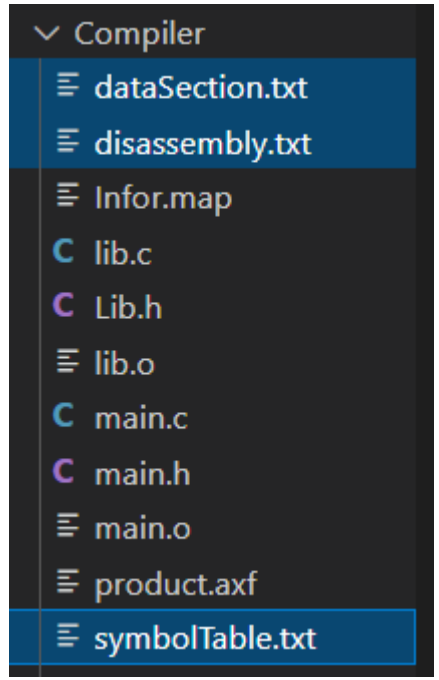
2. Fromelf - Command-line option

- Fromelf command line option:
  - -d: print contents of data section
  - Command line:
    ./fromelf --text -d C:/cygwin/cygwin/home/Compiler/product.axf -o C:/cygwin/cygwin/home/Compiler/dataSection.txt
  - -s: print symbol table
  - Command line:
    ./fromelf --text -s C:/cygwin/cygwin/home/Compiler/product.axf -o C:/cygwin/cygwin/home/Compiler/symbolTable.txt

  - We can try the options shown below:

```
Flags for Text Information
-v          verbose
-a          print data addresses (For images built with debug)
-c          disassemble code
-d          print contents of data section
-e          print exception tables
-g          print debug tables
-r          print relocation information
-s          print symbol table
-t          print string table
-y          print dynamic segment contents
-z          print code and data size information
```

- The result:



2. Fromelf - Command-line option

- Convert .axf file to another format:

```
Binary Output Formats:
       --bin           Plain Binary
       --m32           Motorola 32 bit Hex
       --i32           Intel 32 bit Hex
       --vhx           Byte Oriented Hex format
```

- Example: Convert product.axf to product.hex
  - Command line: ./fromelf --i32 C:/cygwin/cygwin/home/Compiler/product.axf –o J:/Output/product.hex

```
PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ ./fromelf --i32 C:/cygwin/cygwin/home/Compiler/product.axf -o J:/Output/product.hex

PC@DESKTOP-QEN4LJI /cygdrive/c/Keil_v5/ARM/ARMCLANG/bin
$ cd "J:\Output"

PC@DESKTOP-QEN4LJI /cygdrive/j/Output
$ ;s
bash: syntax error near unexpected token `;'

PC@DESKTOP-QEN4LJI /cygdrive/j/Output
$ ls
product.hex
```


Thank you